

Full length article

Packet-syscall coordinated fuzzing for linux kernel network stack

Guoyu Yin^a, Yuhan Chen^{a,b}, Changjian Liu^a, Weibo Zhang^a, Fan Ding^a, Dalong Shi^c,
Yuheng Shen^d, Heyuan Shi^{a,*}

^a Central South University, Changsha, China

^b State Grid Chongqing Company Party School (Training Center), Chongqing, China

^c AVIC International Digital Network Technology Co., Ltd., Beijing, China

^d Tsinghua University, Beijing, China

ARTICLE INFO

Keywords:

Kernel fuzzing

Network stack

Bug detection

Coordinated fuzzing

ABSTRACT

The Linux kernel network stack is a safety-critical subsystem that manages network communications, such as TCP/IP session handling and UDP datagram routing. However, effectively fuzzing the kernel network stack presents significant challenges: large portions of code are disabled by static configurations; generating network-specific inputs for both syscalls and packets is difficult; and testing the interplay between syscall execution and packet injection requires precise coordination. In this paper, we introduce Pacaller, a packet-syscall coordinated fuzzer designed for the kernel network stack. Pacaller first performs automated configuration analysis to enhance code reachability. Then, Pacaller extracts network resources from the kernel source code to generate network-specific syscall specifications and packet models. Finally, Pacaller coordinates syscall execution with packet injection to test their complex interactions within the network stack. We implemented and evaluated Pacaller on four recent versions of the Linux kernel, discovering 26 previously unknown bugs (16 confirmed, 10 fixed). Pacaller also achieved 13.3% to 22.3% higher network stack coverage than Syzkaller, SyzGen++, KernelGPT, Syz-TCP, and TRON. These results demonstrate that Pacaller significantly improves the effectiveness of fuzzing the kernel network stack.

1. Introduction

As a critical component of the Linux kernel, the network stack underpins all network communication. The stack implements a wide array of protocols, such as TCP/IP and UDP, alongside complex routing mechanisms to support user-space applications ranging from real-time streaming to secure enterprise communications. This complexity, stemming from a vast feature set, intricate state models, and the continuous need to handle edge cases like packet malformations, makes the stack highly susceptible to implementation flaws. Such flaws can lead to severe security bugs, including denial-of-service attacks, privilege escalation, or critical data leaks (Bhurtel and Rawat, 2023). In critical infrastructures where network reliability is paramount, these bugs can disrupt systems, underscoring the urgent need for robust and effective testing methodologies.

Fuzz testing (fuzzing) (Manes et al., 2018; McDonald et al., 2021; Shi et al., 2025, 2024; Shen et al., 2024) has become a cornerstone for identifying kernel bugs due to its effectiveness and scalability. When applied to the kernel network stack, testing methodologies diverge into two distinct approaches. State-of-the-art kernel fuzzers (Icamtuf, 2013;

Jeong et al., 2019; Zhang et al., 2024; Shen et al., 2021; Xu et al., 2020; Wang et al., 2021; Vyukov and Konovalov, 2015a; Yang et al., 2025), such as Syzkaller (Vyukov and Konovalov, 2015a), employ system calls (syscalls) to emulate kernel behavior and manage network resources. In a separate domain, protocol fuzzers (Tech, 2013; Fiterau-Brosteau et al., 2020; Wang et al., 2024; Zheng et al., 2019; Eisele et al., 2023), such as Tech (2013), inject sequences of network packets to test traffic processing logic.

However, the effectiveness of these approaches is limited when applied in isolation, as neither can independently address the complexities of the network stack. Syscall-based fuzzers cannot generate detailed protocol information required to probe deep network logic. Conversely, protocol fuzzers operate from user space and cannot coordinate the specific kernel states required to trigger many bugs. This gap means that bugs arising from the intricate interplay between syscall execution and packet injection are often overlooked, leaving significant portions of the kernel network stack undertested. While approaches like TCP-Fuzz (Zou et al., 2021) and EmNetTest (Amusuo et al., 2023) have tried

* Corresponding author.

E-mail addresses: yinguoyu@csu.edu.cn (G. Yin), Chenyuhan@csu.edu.cn (Y. Chen), driz2t@foxmail.com (C. Liu), zhangweibo@csu.edu.cn (W. Zhang), dingdang66686@csu.edu.cn (F. Ding), shidl@avic.com (D. Shi), syh1308@gmail.com (Y. Shen), hey.shi@foxmail.com (H. Shi).

<https://doi.org/10.1016/j.cose.2026.105141>

Received 16 March 2026; Received in revised form 27 May 2026; Accepted 1 September 2026

Available online 11 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

to combine both methods, they are designed for specific protocols like TCP or for different platforms such as embedded systems, and are not directly applicable to the kernel network stack.

To effectively fuzz the Linux kernel network stack, we must address the following challenges:

(1) Identifying network-related static configurations. Significant portions of network code are disabled by default. Activating them requires resolving intricate dependencies among thousands of configuration options. For example, to test the NetEm scheduler, the `CONFIG_NET_SCH_NETEM` option must be enabled, which in turn depends on other options like `CONFIG_NET_SCHED`.

(2) Generating network-specific fuzzing inputs. Generating effective inputs is also challenging. For syscalls, many network protocols lack dedicated interfaces and generating protocol-aware specifications for them is difficult due to their inherent complexity and vast state space. For packets, probing complex logic such as packet reassembly requires a semantic awareness of protocol formats and stateful interactions, which is a capability beyond generic mutation strategies.

(3) Packet-syscall coordinated fuzzing. Coordinating packet injections with syscall execution is particularly difficult. These input types operate at different semantic levels and require precise coordination to expose bugs hidden in their interplay. For instance, a bug may only be triggered when some specific kernel configurations are established through syscalls, after which a precise packet is sent to probe the kernel in that specific context.

To address these challenges, we introduce Pacaller, a novel packet-syscall coordinated fuzzer, designed to enhance the security of the Linux kernel network stack. In summary, Pacaller prepares network-specific inputs before executing them with a coordinated strategy. First, Pacaller performs automated configuration analysis to expose previously unreachable network stack code. Second, Pacaller extracts network entities such as data structures and constants from the source code to generate both accurate syscall specifications and protocol-aware packet models. Finally, Pacaller employs a packet-syscall coordinated approach to coordinate syscall execution with packet injections. These techniques enable Pacaller to probe complex, context-dependent behaviors within the network stack, uncovering bugs inaccessible to conventional fuzzing approaches.

We have implemented and evaluated Pacaller on Linux kernel v6.11, 6.13, 6.15, and 6.16. Our evaluation shows that Pacaller uncovered 26 previously unknown bugs, with 16 confirmed and 10 fixed by kernel maintainers. Furthermore, Pacaller achieved 13.3% to 22.3% higher network stack coverage than Syzkaller, SyzGen++, KernelGPT, Syz-TCP, and TRON. In summary, our contributions are as follows:

- We propose three techniques for network stack fuzzing: automated configuration analysis, resource-driven input generation, and the coordination of syscall execution with packet injection.
- We present the design and implementation of Pacaller, a packet-syscall coordinated fuzzer for the Linux kernel network stack.
- We conduct a comprehensive evaluation demonstrating that Pacaller discovers 26 previously unknown bugs (16 confirmed, 10 fixed) and improves network stack coverage by 13.3% to 22.3% over state-of-the-art fuzzers.

To facilitate reproducibility, we open source Pacaller at <https://github.com/GuoyuYin/Pacaller>.

2. Background

2.1. Linux kernel network stack

The Linux kernel network stack is a core component of the operating system that manages the processing, routing, and transmission of data packets to support essential protocols like TCP/IP and UDP (Qian and Carpenter, 2012). Its architecture is conceptually aligned with

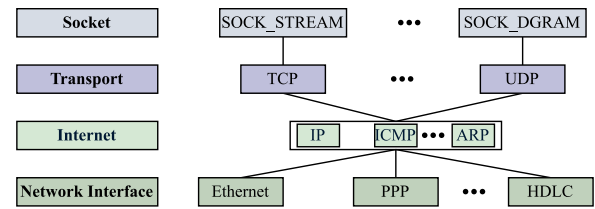


Fig. 1. The layered architecture of the linux kernel network stack.

the TCP/IP model, organizing data flow through a series of protocol layers. This layered design enhances modularity and scalability but also introduces complex inter-layer dependencies, making the stack prone to bugs and highlighting the need for rigorous testing (Bavendiek, 2021).

As shown in Fig. 1, the stack consists of several distinct layers, each with specific roles in packet processing. The network interface layer at the base interacts with physical hardware (e.g., Ethernet) through device drivers. Above the network interface layer, the internet layer, centered on the IP protocol, is responsible for routing packets across networks. The transport layer provides end-to-end communication services, primarily through two means: TCP for reliable, stream-oriented connections, and UDP for low-latency, connectionless datagrams. The socket layer acts as an interface between the user space and the kernel, allowing applications to interact with the transport layer by the sockets API. The security and reliability of the entire stack depend on the correct implementation of each layer and their secure interaction.

2.2. Kernel static configuration analysis

The Linux kernel's high configurability presents a major challenge for testing. A specific configuration determines which code is compiled and thus reachable by a fuzzer. This creates a balance between build time and code coverage. Standard configurations like `defconfig` build quickly but exclude most optional code, leaving large parts of the kernel untested. Conversely, maximal configurations such as `allyesconfig` include much more code but are extremely slow to build, reducing the time available for fuzzing, especially in continuous fuzzing (Hasanov et al., 2025).

To address this problem, research has explored various strategies for generating effective configurations. For example, tools like ConfigFuzz (Zhang et al., 2023) and methods from Yan et al. (2017) apply combinatorial testing to create configurations that cover interactions between options. However, these methods often produce a large number of configurations, which is impractical for kernel fuzzing as each one requires a separate, time-consuming build and fuzzing session. A more targeted approach is configuration repair, represented by tools such as krepair (Yildiran et al., 2024). Instead of creating new configurations, krepair minimally modifies a fast-building base configuration to ensure a given patch is compiled. Krepair uses static analysis and a solver to resolve the complex dependencies within the Kconfig language. Recent work shows configuration repair approach increases bug detection rates in continuous kernel fuzzing (Hasanov et al., 2025). The approach ensures that code changes from new patches are included in the kernel bzImage, which is the executable file that results from kernel compilation.

2.3. Kernel fuzzing and protocol fuzzing

Fuzzing is an automated testing technique that detects bugs by supplying generated inputs to a program, and it has proven highly effective for testing the Linux kernel using syscalls (Vyukov and Konovalov, 2015a). Modern kernel fuzzers are typically coverage-guided, leveraging feedback to prioritize inputs that explore new execution paths, a process often augmented by dynamic analysis tools like KASAN

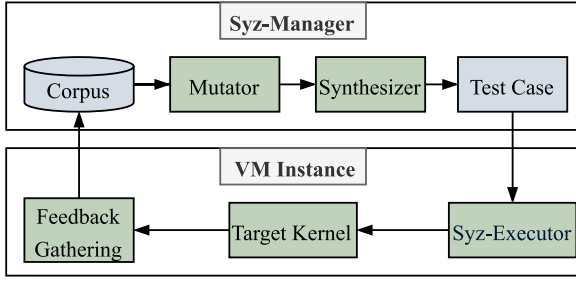


Fig. 2. Overall workflow of Syzkaller.

(Google, 2014) and KCSAN (Google, 2019) to detect memory and concurrency bugs. Among these tools, Syzkaller stands out as the state-of-the-art fuzzer has been integrated into the kernel's continuous integration pipeline to ensure software quality.

In detail, Syzkaller operates based on syscall specifications written in a domain-specific language named Syzlang (Vyukov and Kononov, 2015b). These specifications define kernel interfaces, including data structures, required resources, and related syscalls. Based on Syzlang, Syzkaller generates test cases that simulate the kernel's real-world execution. During fuzzing, as shown in Fig. 2, Syzkaller sends the generated test cases to the target kernel. An executor running within the target kernel then executes the test cases and monitors the kernel's behavior. After execution, Syzkaller collects feedback information, such as code coverage. Based on this feedback, Syzkaller assigns a higher weight to test cases that trigger new system behaviors for further mutation and execution, and discards those that do not. However, Syzkaller's reliance on predefined syscall specifications constrains its coverage of less common or newly introduced kernel behaviors. This limitation has motivated research into automatically generating such specifications. A prominent example is SyzGen++ (Chen et al., 2024), which generates Syzlang specifications from driver binaries by inferring syscall dependencies through techniques like symbolic execution, aiming to enhance kernel code coverage. Additionally, KernelGPT (Yang et al., 2025) leverages Large Language Models (LLMs) to automatically infer Syzlang specifications from kernel source code.

Conversely, network protocol fuzzers specialize in testing packet processing logic. Such tools (Tech, 2013; Luo et al., 2020, 2024; Zuo et al., 2022; Yu et al., 2024) excel at generating complex and often malformed packet sequences based on protocol grammars to uncover bugs. Nonetheless, protocol fuzzers are ill-suited for kernel testing, as their operation in user space prevents them from executing the syscalls required to configure the kernel. Without syscalls to initialize resources like sockets, injected packets fail to reach most of the target code, rendering this approach ineffective for testing the Linux kernel network stack. This methodological separation has led to recent efforts to combine both approaches, yet these solutions are not directly applicable to the Linux kernel network stack.

For instance, TCP-Fuzz (Zou et al., 2021) employs a dependency-based strategy to generate coordinated syscall and packet sequences for testing TCP stacks. However, this approach is unsuitable for general kernel network stack testing because its scope is limited to the TCP protocol and its bug detection relies on differential testing against other stacks. Similarly, EmNetTest (Amusuo et al., 2023) tests embedded network stacks with syscalls to reach protocol states before packet injection. This tool is not transferable to the Linux kernel because embedded network stacks have different architectures and APIs. Other works like Nyx-Net (Schumilo et al., 2022) use snapshot-based fuzzing for high throughput. However, Nyx-Net targets user-space applications by emulating network functions. This design intentionally bypasses the actual kernel network stack to improve performance, leaving kernel-level logic untested. TRON (Zhang, 2025) attempts to test the kernel network stack by synthesizing syscall and packet payloads. However,

```

1 --- a/net/sched/sch_netem.c
2 +++ b/net/sched/sch_netem.c
3 @@ -748,9 +748,9 @@ deliver:
4     if (err != NET_XMIT_SUCCESS) {
5         if (net_xmit_drop_count(err))
6             qdisc_qstats_drop(sch);
7         // Bug: qlen is updated after parent is notified
8         qdisc_tree_reduce_backlog(sch, 1, pkt_len);
9         sch->qstats.backlog -= pkt_len;
10        sch->q.qlen--;
11        qdisc_tree_reduce_backlog(sch, 1, pkt_len);
12    }
13    goto tfifo_dequeue;
14 }

```

Fig. 3. The patch that fixes a use-after-free bug in the NetEm scheduler.

TRON relies on offline trace analysis and static pattern matching to generate inputs. It lacks the dynamic, runtime coordination required to trigger complex state-dependent bugs. Therefore, a methodology that unifies both approaches for the Linux kernel network stack is required.

3. Motivation

In this section, we demonstrate the challenges in fuzzing the Linux kernel network stack by analyzing a motivating example and deriving three core challenges that guide our work.

Motivating Example. We use CVE-2025-21703 (National Vulnerability Database, 2025) as a motivating example, a use-after-free bug in the Linux kernel's NetEm network scheduler. In `net/sched/sch_netem.c`, the flaw arises from an incorrect order of operations within the error-handling path. Specifically, the queue length (`sch->q.qlen`) is decremented after the parent qdisc is notified of a potential queue state change, leading to a brief state inconsistency that can be exploited (lines 8–10). Triggering this bug requires a combination of specific conditions: first, the `CONFIG_NET_SCH_NETEM` static configuration must be enabled, which is disabled in many default kernel builds, rendering the code path unreachable. Second, a user must configure the NetEm qdisc through syscalls and then inject a precise packet that induces a transmission error. This precise interplay between syscall execution and packet injection makes the bug difficult to detect with traditional syscall-based testing. The official patch (as shown in Fig. 3) addresses the flaw by reordering the operations to ensure the queue length is decremented before the parent qdisc is notified (lines 7–11). This change prevents the state inconsistency that could lead to a use-after-free condition.

Challenges. To effectively test the Linux kernel network stack, we must address the following challenges:

(1) **Identifying Network-Related Static Configurations.** The reachability of any given code path within the network stack is conditional upon a specific combination of enabled and disabled static configurations. Identifying the correct combination requires analyzing thousands of kernel options and their intricate dependencies, a task with a worst-case combinatorial complexity of $O(2^n)$, where n is the number of configurable parameters. As highlighted in the motivating example, the vulnerable code path within the NetEm scheduler is guarded by `CONFIG_NET_SCH_NETEM`, which is disabled in most default kernel builds and depends on other options like `CONFIG_NET_SCH`. Without first identifying and enabling such configuration, any fuzzer is incapable of reaching the buggy code.

(2) **Generating Network-Specific Fuzzing Inputs.** Even with the code enabled, generating meaningful fuzzing inputs for the network stack remains a difficult task. This difficulty applies to both syscalls and packets. First, there is a scarcity of dedicated syscall interfaces for many network protocols, and generating protocol-aware syscall specifications is exceedingly difficult due to the inherent complexity and vast state space of these protocols. Second, generating the required malformed or precisely timed packets is a formidable task, as it demands a deep, protocol-specific understanding of data structures and state transitions.

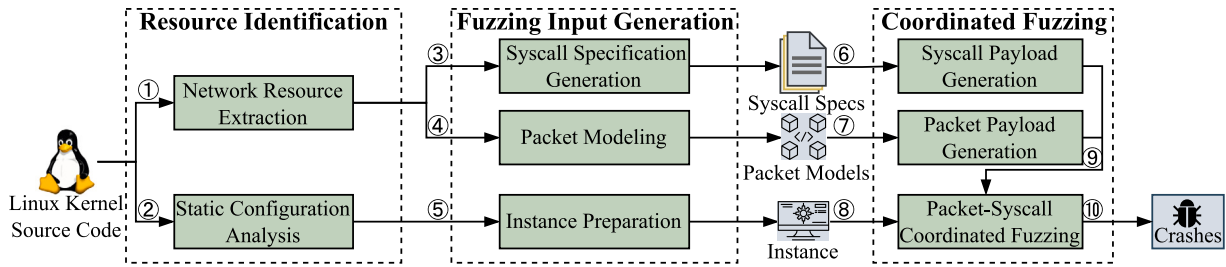


Fig. 4. Overall Workflow of Pacaller. Pacaller processes the Linux kernel source code to ① extract network resource and ② analyze static configurations. The extracted resources are then used to ③ generate syscall specifications and ④ model packets, while the configuration analysis guides ⑤ instance preparation. During fuzzing, the specifications and models are used to generate ⑥ syscall payload and ⑦ packet payload. These payloads are then executed on the prepared ⑧ instance within the ⑨ packet-syscall coordinated fuzzing to find ⑩ crashes.

The motivating example underscores this: a fuzzer must not only generate the correct syscalls to configure the NetEm qdisc but also produce a packet capable of inducing a specific error to trigger the bug.

(3) Packet-Syscall Coordinated Fuzzing. Finally, coordinating low-level packet injections with high-level syscalls is particularly difficult, as they operate at different semantic levels and require precise synchronization to probe for bugs hidden in their complex interplay. As epitomized by CVE-2025-21703, neither step is sufficient in isolation. Triggering the bug requires a synchronized interaction: first establishing a configuration through syscalls (configuring NetEm), and then injecting a precise packet to induce a transmission error that exposes a latent state inconsistency.

4. Design

The overall workflow of Pacaller is illustrated in Fig. 4, comprising three main stages: resource identification, fuzzing input generation, and coordinated fuzzing. In the initial resource identification stage, Pacaller processes the Linux kernel source code to perform two tasks in parallel: extracting network resources for the subsequent input generation and analyzing static configurations to expose more network stack code. Next, in the fuzzing input generation stage, the extracted resources are used to generate syscall specifications and packet models, while the result from the configuration analysis is used for instance preparation. In the final coordinated fuzzing stage, Pacaller uses these generated assets to create syscall and packet payloads, executing them in a coordinated loop against the test instance to efficiently detect crashes.

4.1. Resource identification

Resource identification prepares the kernel for effective fuzzing through two parallel tasks: extracting network resources and analyzing static configurations to improve code reachability.

Network Resource Extraction. Effective network-stack fuzzing requires a deep understanding of its underlying data structures. Therefore, Pacaller employs an automated resource extraction module to identify and extract critical network entities from the kernel source code. The module analyzes the `net`, `drivers/net`, and include directories, as these contain the core definitions for the network stack. This process extracts data structures, type aliases, and semantic constants, enabling the creation of valid and protocol-aware test inputs.

The extraction process is illustrated in Fig. 5. To efficiently identify relevant data structures, Pacaller first performs a keyword-based target filtering. We define a whitelist categorized into three dimensions, as detailed in Fig. 6:

- Base Protocols (e.g., `tcp`, `ipv4`) target fundamental communication logic.
- Core Subsystems (e.g., `netfilter`, `sched`) handle complex packet processing.

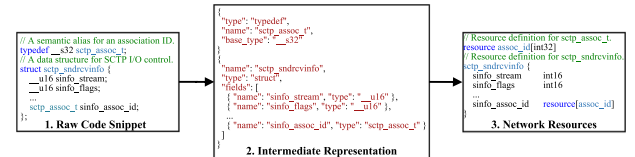


Fig. 5. Example of network resource extraction process.

```
1 CORE_KEYWORDS = [
2     // Base Protocols
3     "socket.h", "sock.h", "in.h", "in6.h", "tcp.h", "udp.h",
4     "icmp", "igmp", "ipv4", "ipv6",
5
6     // Core Subsystems
7     "netfilter", "nf_", "xt_", // Firewall/NAT
8     "sched/sch_", "pkt_sched", // QoS Control
9     "xfrm", "ipsec", // IPsec Security
10    "bridge", "vlan", // L2 Switching
11    "neighbor", "rtnetlink", // Routing/Neighbor
12    "ethtool", // NIC Config
13
14    // Extended Protocols
15    "sctp", "dccp", "l2tp", "rds", "tipc", // Transport
16    "can", "bluetooth", "hci", "l2cap", "rfcomm", // Vehicle/BT
17    "ax25", "rose", "netrom", // Radio
18    "wireless", "cfg80211", "mac80211", "nl80211", // Wi-Fi Stack
19    "mptcp", "unix", "netlink", "mpls", "gre" // Tunnel
20 ]
```

Fig. 6. The keyword whitelist used for filtering target network files.

- Extended Protocols (e.g., `sctp`, `can`) cover specialized network stacks in the `net/` directory.

This process identifies over 550 high-value target files from the kernel source code, varying slightly across kernel versions.

Next, Pacaller analyzes these files using an AST-based approach. We first preprocess the code to expand kernel macros (e.g., `__u32`) into standard C types to resolve type ambiguities. Then, Pacaller parses the code into an Abstract Syntax Tree (AST). The extractor traverses the AST to collect field names, types, and array sizes into an Intermediate Representation (IR). For example, as shown in Fig. 5, Pacaller parses the AST node for `sctp_sndrcvinfo` and converts the structure into the IR format. This IR is subsequently transformed into a resource definition, a process that resolves dependencies by converting implicit type usages into explicit links between resources. This automated process yields a rich set of network resources used in the next phase to generate syscall specifications and packet models.

Through this filtering and extraction mechanism, Pacaller is capable of supporting various network protocols. Specifically, based on the extracted resources and protocol definitions, Pacaller supports generating test cases for 35 distinct protocols listed in Table 1. Note that we count ICMP and ICMPv6 as a single protocol type. These protocols cover the primary network and transport layers, along with critical wireless and tunneling protocols, constituting the main attack surface of the kernel network stack.

Table 1
The 35 network protocols supported by Pacaller.

Category (Layer)	Protocols
Core Protocols (Network)	IPv4, IPv6, ICMP/ICMPv6, IGMP, GGP, ST, CBT, PUP, IDP
Data Delivery (Transport)	TCP, UDP, UDPLITE, SCTP, DCCP, TP, MTP
Routing Control (Network)	OSPF, VRRP, RSVP, PIM, EGP
Tunnel & Security (Net/Interface)	GRE, ESP, AH, IPIP, L2TP, MPLS, IPComp, BEETPH, ENCAP
Specialized Stacks (Interface/Trans)	Bluetooth, 802.11, CAN, TIPC, RDS

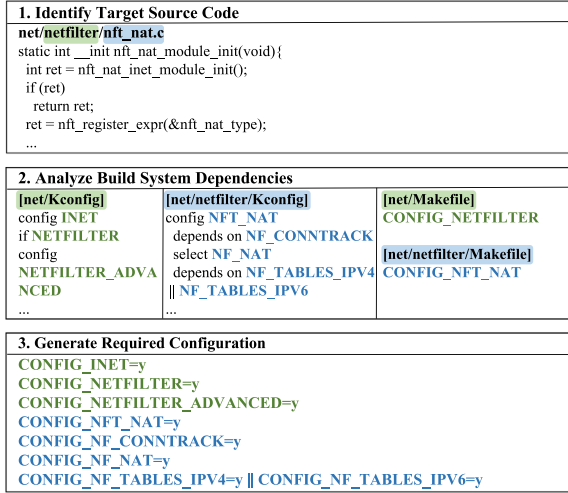


Fig. 7. Example of static configuration analysis. The analysis resolves build dependencies for a target file (nft_nat.c) to generate a valid compilation configuration.

Static Configuration Analysis. In the highly configurable kernel, the inclusion of specific code paths during compilation is dictated by numerous static configuration options. The default configurations often disable many optional features to prioritize core functionality and reduce build times. This practice limits the testability of the network stack, as large portions of its code, such as the Network Address Translation (NAT), controlled by CONFIG_NFT_NAT, remain uncompiled and thus unreachable by the fuzzer. This exclusion of code during the build process creates critical gaps in test coverage, potentially leaving latent bugs undetected.

To address this challenge, Pacaller employs a static analysis method. As detailed in Fig. 7, the analysis begins with target source files in the net/ and drivers/net/ directories. Pacaller first parses Makefiles to map these files to their corresponding CONFIG_* options. Pacaller then parses Kconfig files to collect relations among options, such as depends on and select, to construct a dependency graph. To resolve the complex constraints in this graph, Pacaller utilizes krepair (Yildiran et al., 2024), a tool that iteratively repairs a .config file until all presence conditions for the target set are satisfied. When encountering conflicting options, Pacaller employs a coverage-based strategy. We correlate the priority of each option with the lines of associated source code. The solver selects the option branch that enables more lines of code. Through this maximization strategy, we generate a single configuration containing the maximal set of compatible network features, thereby avoiding the overhead of maintaining multiple kernel images. This yields a minimal configuration for compiling the target network stack.

We chose this targeted approach over using a standard allyesconfig. The allyesconfig option compiles many unrelated subsystems, which increases build time and dilutes the fuzzing effort on the network stack. In contrast, our generated configuration is optimized for fuzzing the network stack. Notably, our method generates a single configuration because mutually exclusive code paths within #if/#else blocks are uncommon in the core network stack logic.

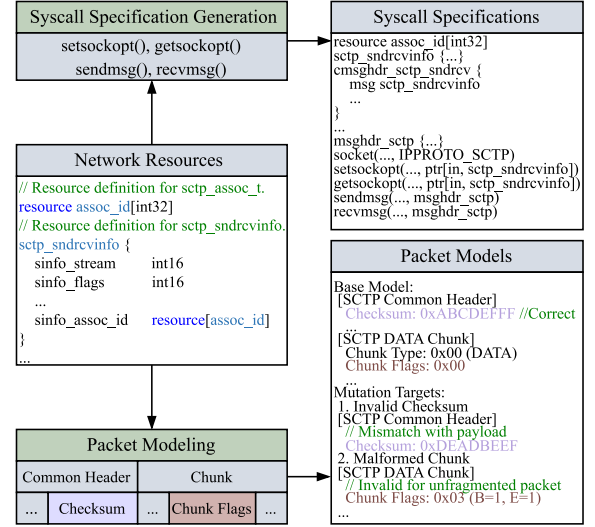


Fig. 8. Example of fuzzing input generation process. In this stage, extracted resources are transformed into syscall specifications and packet models.

4.2. Fuzzing input generation

The fuzzing input generation stage transforms the extracted network resources into two distinct classes of artifacts for fuzzing. As illustrated in Fig. 8, this stage involves two parallel generation processes. The first process generates protocol-aware syscall specifications designed to test kernel logic through sequences of syscalls. The second process constructs packet models to evaluate the kernel's packet handling logic. This dual-generation strategy supplies the necessary inputs for the subsequent coordinated fuzzing stage.

Syscall Specification Generation. Syscall specification generation translates the extracted kernel entities into Syzlang specifications, enabling the fuzzer to generate protocol-aware test payloads. As illustrated in Fig. 8, this process models a kernel data structure, such as sctp_sndrcvinfo, as a formal definition that can be used as an argument to setsockopt or as ancillary data in sendmsg. The generation relies on two main inputs:

- **Templates** are protocol-specific Syzlang definitions that outline relevant syscalls. We constructed protocol-specific templates for 35 distinct protocols, as listed in Table 1. Creating these templates was a one-time manual effort that took approximately one day. The templates define standard socket operations like socket, setsockopt, and sendmsg. For example, the SCTP template provides signatures for setsockopt() and sendmsg().
- **Predefined Primitives** are common Syzlang elements shared across all specifications, such as include directives and basic resource definitions like resource sock_in6[sock].

Algorithm 1 formalizes this process. The algorithm takes a set of extracted kernel network resources R , a collection of templates T , and predefined primitives PT as input. First, the algorithm partitions the resources into groups G based on their subsystem, such as SCTP (line

2). For each group, the process selects a protocol-specific template and iterates through each resource (lines 4–6). To link syscalls with data structures, Pacaller maps extracted structures into the syscall arguments defined in the template. In the Linux kernel, syscalls like `setsockopt` determine their argument type based on the protocol level (e.g., `SOL_SCTP`). Specifically, the template defines the data argument (e.g., `optval`) as a Syzlang union, representing a collection of candidate types. The `Generate` function (line 7) converts each resource into a Syzlang construct and aggregates it into this union within the protocol specification. By placing these resources within the same protocol scope, they become valid candidate arguments for the syscalls defined in the template. This approach ensures that the fuzzer passes the correct structure type to the syscall during runtime testing. Crucially, after all resources are processed, `ResolveDeps` analyzes these constructs to establish a valid declaration order by resolving inter-resource dependencies (line 8). This function constructs a dependency graph where nodes are data structures and edges represent field references. A topological sort is then performed on this graph. For example, this step ensures that the `assoc_id` resource is defined before the `sctp_sndrcvinfo` struct that references it. Finally, the `Assemble` function (line 9) combines these ordered constructs with the predefined primitives to form the complete specification file (lines 10–12), as shown in Fig. 8. These generated specifications allow the fuzzer to test kernel logic that operates on complex network-specific data structures.

Algorithm 1: Syscall Specification Generation

Input: Resources R ; templates T ; predefined primitives PT
Output: Syzlang specification files S

```

1  $S \leftarrow \emptyset$ 
2  $G \leftarrow \text{Group}(R)$  // Partition by subsystem
3 for each protocol  $p$  and its resources  $R_p$  in  $G$  do
4    $t \leftarrow \text{SelectTemplate}(p, T)$ ;
5    $\Sigma_{\text{raw}} \leftarrow \emptyset$ ;
6   for each resource  $r \in R_p$  do
7      $\Sigma_{\text{raw}} \leftarrow \Sigma_{\text{raw}} \cup \text{Generate}(r, t)$  // Generate Syzlang definitions
8    $\Sigma_{\text{ord}} \leftarrow \text{ResolveDeps}(\Sigma_{\text{raw}})$  // Order by dependency
9    $\Sigma_{\text{final}} \leftarrow \text{Assemble}(PT, \Sigma_{\text{ord}})$ ;
10   $f_{\text{name}} \leftarrow \text{GenFileName}(p)$ ;
11   $\text{Write}(\Sigma_{\text{final}}, f_{\text{name}})$ ;
12   $S \leftarrow S \cup \{f_{\text{name}}\}$ 
13 return  $S$ 
```

Packet Modeling. Packet modeling complements syscall-based testing by generating direct packet-level inputs to probe the kernel's packet handling routines. This process leverages the extracted network resources to construct grammar-based models for protocol packets. As illustrated in Fig. 8, the process begins by establishing a well-formed base model, such as an SCTP packet complete with a common header and data chunk. This base model then serves as a template for applying mutations using operators.

We define packet templates in JSON format to represent the protocol structure. Fig. 9 presents the JSON template for the SCTP protocol, corresponding to the standard SCTP packet format. The template defines the Common Header, which includes the Source Port, Destination Port, Verification Tag, and Checksum. Following the header, the template defines a list of Chunks, where each chunk consists of Chunk Type, Chunk Flags, Chunk Length, and Chunk Value. Templates contain placeholders for these fields. We constructed templates for 35 distinct protocols, as listed in Table 1. Creating these templates was a one-time manual effort that took approximately 10 h.

Mutation operators are predefined, grammar-based transformations. Each operator is a tuple (f_{target}, op) that specifies a field to mutate and the mutation primitive to apply. The operation op defines the specific mutation action, which falls into several categories:

```

1 {
2   "protocol": "SCTP",
3   "structure": [
4     { "name": "Common Header", "fields": [
5       { "name": "Source Port Number", "type": "uint16" },
6       { "name": "Destination Port Number", "type": "uint16" },
7       { "name": "Verification Tag", "type": "uint32" },
8       { "name": "Checksum", "type": "uint32" }
9     ]
10    },
11    { "name": "Chunks", "type": "list", "element": {
12      "fields": [
13        { "name": "Chunk Type", "type": "uint8" },
14        { "name": "Chunk Flags", "type": "uint8" },
15        { "name": "Chunk Length", "type": "uint16" },
16        { "name": "Chunk Value", "type": "blob", "size":
17          "var" }
18      ]
19    }
20  ]
21 }
```

Fig. 9. The template describing the structure of an SCTP packet.

- **Value Replacement:** Replaces the field's current value with special values (e.g., 0, -1, `MAX_INT`), boundary values, or random data.
- **Bit Manipulation:** Performs bitwise operations on flag fields, such as setting or clearing specific bits, or applying a mask to create logically invalid combinations.
- **Size Modification:** Alters the value of fields that represent length or size to mismatch the actual size of the packet data.
- **Structural Modification:** Duplicates, removes, or reorders optional protocol components (e.g., header options).

For instance, one operator could target the checksum field and replace it with an incorrect value. Another operator might target the SCTP chunk flags and apply a transform that sets a logically invalid combination, such as marking a single chunk as both the beginning (`B flag` = 1) and the end (`E flag` = 1) of a fragment. These models provide the foundation for evaluating the kernel's packet processing logic and uncovering bugs that are unreachable through syscall execution alone.

Algorithm 2: Packet Modeling

Input: Resources R ; templates T ; mutation operators M
Output: Base packet corpus P_{base} ; mutated packet p_{var}

```

1 Function  $\text{GenerateBaseCorpus}(R, T)$ 
2    $P_{\text{base}} \leftarrow \emptyset$ ;
3   for each resource  $r \in R$  do
4      $p \leftarrow \text{Proto}(r)$  // Extract protocol
5      $t \leftarrow \text{SelectTemplate}(p, T)$ ;
6      $p_{\text{base}} \leftarrow \text{Construct}(r, t)$  // Fill template with valid constants
7      $P_{\text{base}} \leftarrow P_{\text{base}} \cup \{p_{\text{base}}\}$ ;
8   return  $P_{\text{base}}$ 
9 Function  $\text{Mutate}(P_{\text{base}}, M)$ 
10   $\mu \leftarrow \text{SelectOperator}(M)$  // Select one operator
11   $p_{\text{var}} \leftarrow \text{Copy}(P_{\text{base}})$  // Deconstruct operator
12   $(f_{\text{target}}, op) \leftarrow \mu$ ;
13   $\text{ApplyTransform}(op, p_{\text{var}}, f_{\text{target}})$ ;
14  return  $p_{\text{var}}$ 
```

Algorithm 2 outlines the two core functions of the packet modeling process. The algorithm takes extracted kernel network resources R , a set of templates T , and a predefined set of grammar-based mutation operators M as input. The first function, `GenerateBaseCorpus`, creates the initial base corpus by iterating through the extracted resources. For each resource, the `Proto` function (line 4) extracts the protocol type, `SelectTemplate` (line 5) chooses a corresponding template. The `Construct` function (line 6) then fills the template by populating its fields with the valid constants defined in the resource r . For fields that match kernel constants, such as `Chunk Type`, Pacaller assigns

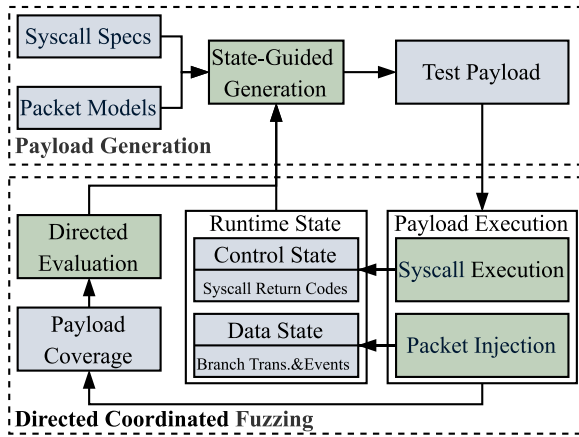


Fig. 10. The coordinated fuzzing process of Pacaller. The process couples payload generation guided by runtime state with directed coordinated fuzzing guided by coverage feedback.

the extracted constant `SCTP_DATA` (value 0). For fields without fixed values, such as `Chunk Value`, Pacaller fills them with random data. This step ensures that Pacaller generates valid base packets for all 35 protocols. The second function, `Mutate`, defines the strategy for dynamic generation of packet variants. This strategy is guided by the mutation operators in M , where each operator μ is a tuple (f_{target}, op) that specifies a field to mutate and the mutation primitive to apply. Pacaller invokes this function during its execution loop. The function first uses `SelectOperator` (line 11) to choose a mutation operator, and then applies the transform to a copied packet using `ApplyTransform` (line 13) to produce a variant. This grammar-based approach focuses on the structural and semantic validity of individual packets, while the complex task of coordinating them with syscall execution to test state-dependent behaviors is handled by the coordinated fuzzing stage.

4.3. Coordinated fuzzing

The coordinated fuzzing stage constitutes the core execution engine of Pacaller, managing the fuzzing process as shown in Fig. 10. This stage operates through two tightly coupled processes: payload generation and directed coordinated fuzzing. These processes are interconnected within a feedback-driven loop. The payload generation process employs state-guided generation, leveraging kernel runtime information to create context-aware test payloads. Concurrently, the directed coordinated fuzzing process uses a directed evaluation mechanism, which utilizes coverage feedback to steer the fuzzing effort toward high-value target areas, namely the `net/` and `drivers/net/` directories. This integrated approach enables Pacaller to probe complex, state-dependent behaviors, uncovering previously unreachable bugs.

Payload Generation. Payload generation is guided by the kernel runtime state (C_t , D_t).

- The Control State, C_t , captures observable outcomes from syscall execution, primarily represented by syscall return codes. A new return code indicates that the execution result of a syscall has changed. Examples include file descriptors returned by `socket()` or a success code returned by `connect()`. Notably, a successful return code implies that the corresponding parameter configuration has been established in the kernel.
- The Data State, D_t , captures internal kernel events. It consists of two parts: the number of branch transitions triggered by packets and critical kernel semantic events monitored via eBPF probes. A new branch transition implies that the packet has explored new protocol logic or states. Examples include a transition in the TCP state machine or an increment in exception counters.

For example, after a `setsockopt()` call configures the MTU and a packet injection increments a retransmission counter, the state becomes $\langle \{mtu: 1500\}, \{tcp_retrans: 1\} \rangle$. This state establishes a bidirectional feedback loop between payload execution and generation.

(1) Syscall-to-packet guidance. Information derived from syscall execution, which constitutes the control state, guides packet-level probing. Specifically, the execution results of syscalls determine the packet generation strategy. For instance, a successful return code from a `setsockopt()` call (implying the configuration is established) causes the payload generator to prioritize creating packets that match this configuration to test fragmentation logic. This guided approach enables the generation of packets that test specific conditions established by syscalls. We use the interaction score as an evaluation metric. If a constructed packet alters the return codes of syscalls (e.g., from error to success), this packet is considered effective.

(2) Packet-to-syscall guidance. Conversely, events from packet injection, which constitute the data state, guide syscall-level exploration. The data state D_t , updated after packet injection, guides the syscall selection strategy. Internal kernel events, such as an incremented TCP retransmission counter, are not directly visible from userspace. To capture these signals, we employ a lightweight monitor implemented with eBPF. This monitor is designed for minimal overhead; we attach probes only to 19 critical functions covering packet entry points, protocol layers, and state events. The efficiency of eBPF's in-kernel compilation, combined with our selective instrumentation, ensures that the impact on fuzzing throughput (executions-per-second) is negligible. The monitor reports these internal state changes to the generation process via efficient eBPF maps. This feedback enables the fuzzer to react to network events. For example, when packet injection produces a data state indicating protocol state transitions or repeated retransmissions, the fuzzer prioritizes syscalls such as `close()` to test connection error-handling paths. This approach enriches syscall-level exploration by enabling it to respond to relevant data state changes. If subsequent packets trigger more branch transitions (part of D_t), the fuzzer prioritizes that syscall sequence.

Directed Coordinated Fuzzing. The directed coordinated fuzzing process is where inputs are executed and their feedback is processed. A key component of this process is the directed evaluation mechanism, which concentrates fuzzing resources on the kernel network stack by implementing a weighted scoring strategy for the payload coverage. This approach addresses a common inefficiency in general fuzzers, which often dilute their efforts on irrelevant subsystems by treating all new code coverage equally.

Our evaluation method prioritizes feedback from inputs probing the network stack. For a test input I , the score is:

$$S(I) = |\text{Cov}_{\text{new}}(I)| + \lambda \cdot |\text{Cov}_{\text{new}}(I) \cap T| \quad (1)$$

Here, $\text{Cov}_{\text{new}}(I)$ is the set of new basic blocks triggered by input I , and T is the predefined set of target blocks within the `net/` and `drivers/net/` directories. The first term maintains a baseline incentive for broad exploration, while the second provides a scaled reward for discovering paths within our target regions, controlled by the hyper-parameter λ .

This score guides the fuzzer's input selection strategy. An input that achieves a higher score is prioritized for subsequent testing. This process consistently directs the fuzzing effort toward exploring the network stack.

Algorithm 3 details the complete coordinated fuzzing loop, which integrates the payload generation and directed coordinated fuzzing strategies. To prioritize effective inputs, Pacaller maintains a global variable `global_max_score` to track the highest interaction level observed (line 10). The function `Coordinate` (lines 1–4) mixes generated inputs by combining new packets with previous syscalls and pairing new syscalls with previous packets. The function `CalcInteractionScore` (lines 5–9) calculates the interaction score. This score is the product of

Algorithm 3: Coordinated Fuzzing Loop

Input: Syscall Specs S ; Packet Models P
Output: Crashes

```

1 Function Coordinate( $sys_{new}, pkt_{new}, ctx$ )
2    $prog_A \leftarrow \text{Combine}(pkt_{new}, ctx.sys_{last})$ ;
3    $prog_B \leftarrow \text{Combine}(sys_{new}, ctx.pkt_{last})$ ;
4   return ( $prog_A, prog_B$ )
5 Function CalcInteractionScore( $C_{ret}, cnt_{trans}$ )
6    $cnt_{ret} \leftarrow \text{CountUnique}(C_{ret})$ ;
7   // Logarithmic scaling to dampen noise
8    $impact_{pkt} \leftarrow \log_2(1 + cnt_{trans})$ ;
9    $score \leftarrow cnt_{ret} \times impact_{pkt}$ ;
10  return  $score$ 
11 ( $Corpus, Crashes, global\_max\_score$ )  $\leftarrow$  (InitCorpus(),  $\emptyset, 0$ );
12 while True do
13   // Step 1: Context Awareness
14   ( $ctx, state$ )  $\leftarrow$  FetchContextAndState( $Corpus$ );
15   // Step 2: Generate and Coordinate
16   ( $sys_{new}, pkt_{new}$ )  $\leftarrow$  GenerateTestPayloads( $S, P, state$ );
17    $progs \leftarrow \text{Coordinate}(sys_{new}, pkt_{new}, ctx)$ ;
18   // Step 3: Execute and Evaluate
19   for  $prog \in progs$  do
20      $Crashes \leftarrow \text{Execute}(prog)$ ;
21     ( $cov_{new}, C_{ret}, D_{events}, cnt_{trans}$ )  $\leftarrow$  CollectFeedback();
22      $score_{dir} \leftarrow \text{CalcDirectedScore}(cov_{new})$ ;
23      $score_{inter} \leftarrow \text{CalcInteractionScore}(C_{ret}, cnt_{trans})$ ;
24      $prio \leftarrow \text{CalcPriority}(score_{dir}, score_{inter})$ ;
25     if  $score_{dir} > 0$  or  $score_{inter} > global\_max\_score$  or  $IsUnseen(D_{events})$  then
26       if  $score_{inter} > global\_max\_score$  then
27          $global\_max\_score \leftarrow score_{inter}$ ;
28       SaveToCorpus( $Corpus, prog, prio$ );
29 Output( $Crashes$ );

```

the diversity of syscall return codes and the number of branch transitions triggered by packets. We apply logarithmic scaling to balance syscall return codes and packet-triggered transitions:

$$Score_{inter} = cnt_{ret} \times \log_2(1 + cnt_{trans}) \quad (2)$$

where cnt_{ret} denotes the count of unique syscall return codes, and cnt_{trans} represents the number of branch transitions triggered by packet injection. This formula prevents high-frequency packet events from dominating the score. In the execution phase (line 16), Pacaller utilizes the TUN/TAP interface to inject raw Ethernet frames. We employ a multi-threaded architecture where a dedicated thread handles packet injection alongside the main syscall thread, coordinated by shared memory flags and spinlocks. The eBPF module captures non-control-flow state changes (D_{events}) (line 17), such as protocol state transitions, which form the remaining part of the data state (D_r). After execution, Pacaller calculates the priority ($prio$) for the input. Both the directed score ($score_{dir}$) and the interaction score ($score_{inter}$) contribute to the priority ($prio$) (line 20). Finally, the algorithm employs a hybrid selection strategy to update the corpus based on three criteria: generating new code coverage ($score_{dir} > 0$), exceeding the highest recorded interaction score ($score_{inter} > global_max_score$), or triggering previously unseen semantic events (D_{events}) (line 21). If any of these criteria is satisfied, Pacaller saves the program using the calculated priority ($prio$) (line 24). A record interaction score updates $global_max_score$ to guide generation (lines 22–23).

5. Implementation

Our implementation of Pacaller is written in C, Go, and Python, extending the Syzkaller architecture with custom components for Linux kernel network stack fuzzing.

Resource Identification. We use the Clang LibTooling interface to parse the kernel's C source code into an AST, from which we extract critical network entities such as data structures and constants. Concurrently, a Python-based module parses `Kconfig` and `Makefile` files to build a dependency graph. We use `krepair` to solve this graph and generate a minimal valid `.config` for network stack fuzzing.

Fuzzing Input Generation. The extracted resources are serialized into a JSON-based IR. A Python engine transforms this IR into protocol-aware syscall specifications and abstract packet models, which form the initial fuzzing corpus.

Coordinated Fuzzing. Our coordinated fuzzing engine operates between the guest VM and the host machine. Inside the guest VM, a custom C executor launches a dedicated thread for concurrent packet injection through the TUN/TAP interface alongside syscall execution. We use shared memory flags and spinlocks to synchronize the packet injection with the syscall execution. To capture critical kernel semantic events for the data state, Pacaller employs eBPF probes to monitor 19 kernel functions as listed in Table 2. These functions cover packet entry points, protocol layers, state transitions, and packet lifecycles. For instance, `tcp_set_state()` tracks protocol state changes, while `kfree_skb_reason()` captures packet drops. The monitor reports these events to the manager to guide the coordinated fuzzing. On the host, a Go-based manager analyzes feedback from the guest, including code coverage and runtime state information. The manager applies our weighted scoring function to this feedback to guide new input generation, steering the fuzzing effort toward the kernel network stack.

6. Evaluation

We evaluate Pacaller through four research questions:

- **RQ1:** How effective is Pacaller at detecting bugs in the kernel network stack, and what are the specific contributions of different components?
- **RQ2:** How does Pacaller compare with state-of-the-art kernel network stack fuzzers?
- **RQ3:** How significant is the performance gain from the network-specific syscall specifications, packet injection, and the coordinated fuzzing strategy?
- **RQ4:** How does λ affect Pacaller's fuzzing performance?

6.1. Evaluation setup

We evaluated Pacaller on four recent versions of the Linux kernel: v6.11, v6.13, v6.15, and v6.16. Linux v6.16 was the latest stable release at the time of our experiments. All kernels were compiled using the optimized configurations produced by our static configuration analysis, with `CONFIG_KCOV` (SimonKagstrom, 2015) enabled for code coverage collection and `CONFIG_KASAN` for memory errors detection.

Our experiments were conducted on a high-performance server equipped with a 256-core AMD EPYC 7742 CPU and 256 GiB of memory. Our evaluation focuses on discovering unknown bugs rather than using a known bug benchmark. A standardized bug benchmark for kernel fuzzing is impractical, as the rapid evolution of the Linux kernel would quickly render any fixed set of bugs obsolete. For bug detection, Pacaller was run continuously for one week on each kernel version. For other comparisons, each fuzzer was executed under identical QEMU (Bellard, 2005) configurations (2 VMs, with 2 cores for each fuzzing instance) for 24 h and repeated 10 times, following the fuzzing evaluation best practice (Klees et al., 2018).

6.2. Bug detection effectiveness

To answer RQ1, we deployed Pacaller to fuzz each target kernel for one week to uncover previously unknown bugs within the network stack. We further analyzed the specific contribution of each component to the discovered bugs. Specifically, we categorized the bugs into four types based on the trigger source, as shown in the "Trigger." column of Table 3. Baseline Syscalls (BC) denotes bugs triggered only by the baseline syscall specifications in Syzkaller. Network-specific Syscalls (NC) denotes bugs triggered only by the network-specific syscall specifications generated by Pacaller. Packet (PC) denotes bugs triggered only by packet injection. Coordination (CD) denotes bugs triggered only by the coordination between syscalls and packets. Additionally, we evaluated the impact of the static configuration analysis, recorded in the "Config." column, by determining whether each bug required the

Table 2
The 19 kernel functions monitored by Pacaller via eBPF.

Category	Function name
Entry point	netif_receive_skb(), __netif_receive_skb_core()
Network layer	ip_rcv(), ip_local_deliver(), ip6_rcv(), ip6_input()
Transport Layer	tcp_v4_rcv(), tcp_v6_rcv(), udp_rcv(), udpv6_rcv(), sctp_rcv()
Control protocols	icmp_rcv(), icmpv6_rcv()
State & Events	tcp_set_state(), tcp_retransmit_skb(), tcp_drop()
Queue & Lifecycle	sock_queue_rcv_skb(), consume_skb(), kfree_skb_reason()

Table 3
Previously unknown bugs detected by Pacaller.

Idx	Source file	Bug function	Bug type	Status	Trigger.	Config.
1	net/ipv4/ipmr.c	ipmr_free_table()	kernel bug	Fixed	NC	Gen
2	net/ipv6/ip6mr.c	ip6mr_free_table()	kernel bug	Fixed	NC	Gen
3	net/unix/af_unix.c	unix_release()	kernel bug	Reported	BC	Def
4	net/unix/af_unix.c	unix_stream_connect()	kernel bug	Reported	BC	Def
5	net/ipv4/inet_timewait_sock.c	inet_twsk_kill()	kernel bug	Reported	CD	Def
6	net/ipv6/ip6_input.c	ip6_list_rcv()	kernel bug	Reported	PC	Def
7	net/xfrm/xfrm_policy.c	xfrm_policy_timer()	kernel bug	Reported	BC	Def
8	net/ipv6/addrconf.c	ip6_add_addr()	kernel bug	Fixed	CD	Def
9	net/netfilter/nf_conntrack_core.c	nf_ct_alloc_hashtable()	kernel bug	Fixed	CD	Def
10	net/core/sock.c	sock_kmalloc()	kernel bug	Fixed	NC	Def
11	net/sched/sch_skbprio.c	skbprio_enqueue()	kernel bug	Fixed	CD	Gen
12	net/9p/trans_fd.c	p9_fd_create_tcp()	deadlock	Confirmed	CD	Def
13	net/unix/af_unix.c	unix_seqpacket_sendmsg()	deadlock	Confirmed	CD	Def
14	net/ipv4/ip_input.c	ip_rcv()	deadlock	Reported	CD	Def
15	net/unix/af_unix.c	unix_stream_connect()	deadlock	Reported	CD	Def
16	drivers/net/ethernet/intel/e1000/e1000_main.c	e1000_watchdog()	deadlock	Reported	CD	Gen
17	net/9p/client.c	p9_client_destroy()	memory leak	Fixed	CD	Def
18	net/ipv6/exthdrs.c	ip6_renew_options()	general protection fault	Fixed	CD	Def
19	net/9p/trans_fd.c	p9_fd_cancelled()	general protection fault	Fixed	CD	Def
20	net/core/datagram.c	_skb_try_rcv_from_queue()	general protection fault	Reported	CD	Def
21	net/ipv6/ip6_fib.c	fib6_walk_continue()	general protection fault	Confirmed	NC	Def
22	net/ipv4/ip_output.c	ip_finish_output2()	general protection fault	Confirmed	CD	Def
23	net/9p/client.c	p9_req_put()	use-after-free	Confirmed	NC	Def
24	net/core/skbuff.c	napi_skb_cache_get()	use-after-free	Confirmed	CD	Def
25	net/netlink/af_netlink.c	_netlink_deliver_tap()	null-ptr-deref	Fixed	CD	Gen
26	net/ipv6/ndisc.c	ndisc_send_skb()	null-ptr-deref	Reported	CD	Def

generated configuration (Gen) or was reproducible under the default configuration (Def). Pacaller discovered 26 previously unknown bugs, with 16 confirmed and 10 fixed by the kernel maintainer, as detailed in Table 3.

Bug Analysis. These bugs are mostly located in critical networking components, such as the `net/ipv6`, `net/ipv4`, and `net/9p` directories, highlighting security risks within core network subsystems. Furthermore, the detected bugs span diverse bug types, including kernel bugs, deadlocks, memory leaks, general protection faults, use-after-free, and null pointer dereferences. These bugs pose severe threats; for example, a kernel bug can lead to a system panic, while a deadlock can render critical network operations unresponsive. The effectiveness of Pacaller in uncovering such a wide array of bugs stems from its three primary contributions. First, its automated static configuration analysis enhances code reachability, exposing previously dormant network code to the fuzzer. Second, its resource-driven input generation produces protocol-aware syscall specifications and packet models, enabling deep, targeted testing of complex logic. Finally, its packet-syscall coordinated fuzzing triggers intricate interactions inaccessible to conventional fuzzers.

We further detail the specific contribution of each component based on the categorization in Table 3. Table 4 presents the distribution of the discovered bugs across different trigger sources and static configurations. We analyzed the trigger sources and static configurations for each bug to quantify the impact of the components. First, regarding the trigger source, Baseline Syscalls (BC) triggered 3 bugs. The resource-driven input generation proved essential, as Network-specific Syscalls (NC) triggered 5 bugs that the baseline specifications missed. Packet (PC) triggered 1 bug. This result is reasonable because the kernel network stack typically requires syscalls to configure the network state.

Table 4
Distribution of detected bugs based on trigger sources and configurations.

Trigger source	Static configuration		Total
	Default (Def)	Generated (Gen)	
Baseline Syscalls (BC)	3	0	3
Network-specific Syscalls (NC)	3	2	5
Packet (PC)	1	0	1
Coordination (CD)	14	3	17
Total	21	5	26

Therefore, the kernel often drops packets that lack the necessary syscall context. Most significantly, Coordination (CD) triggered 17 bugs. These 17 bugs required the precise coordination between syscall execution and packet injection, which confirms that the coordination strategy is the most critical factor in detecting complex network bugs. Second, regarding the static configuration analysis, Generated Configuration (Gen) enabled the discovery of 5 bugs. As shown in Table 4, these 5 bugs (2 from NC and 3 from CD) reside in code paths that are disabled by default, meaning they would remain undetected without the configuration analysis. The remaining 21 bugs were reproducible under the Default Configuration (Def). This result demonstrates that the static configuration analysis expands code reachability, while Pacaller maintains high effectiveness in testing the default network stack.

Case 1: Network-specific Syscalls Triggered Bug. To demonstrate the significance of network-specific syscall specifications, we present a kernel bug (Bug #1) fixed by maintainers, as shown in Fig. 11. This bug resides in the Linux kernel's IPv4 multicast routing module (`net/ipv4/ipmr.c`) and is exposed when specific kernel configurations are applied. The flaw stems from an inadequate sanity check

```

1 --- a/include/linux/mroute_base.h
2 +++ b/include/linux/mroute_base.h
3 @@ -262,6 +262,11 @@ struct mr_table {
4     int mroute_reg_vif_num;
5 };
6 // New function with enhanced check
7 +static inline bool mr_can_free_table(struct net *net)
8 +{
9     return !check_net(net) || !net_initialized(net);
10 +}
11 --- a/net/ipv4/ipmr.c
12 +++ b/net/ipv4/ipmr.c
13 @@ -317,11 +312,6 @@ EXPORT_SYMBOL(ipmr_rule_default);
14 #define ipmr_for_each_table(mrt, net) \
15     for (mrt = net->ipv4.mrt; mrt; mrt = NULL)
16 // Removed: Old insufficient check
17 -static bool ipmr_can_free_table(struct net *net)
18 -{
19     return !check_net(net);
20 -}
21 // Bug is triggered here
22 @@ -437,7 +427,7 @@ static void ipmr_free_table(struct
23     mr_table *mrt)
24 {
25     struct net *net = read_pnet(&mrt->net);
26     WARN_ON_ONCE(!ipmr_can_free_table(net));
27 + WARN_ON_ONCE(!mr_can_free_table(net));
28     timer_shutdown_sync(&mrt->ipmr_expire_timer);

```

Fig. 11. The patch fixes a kernel bug by replacing an inadequate check with a robust alternative.

in `ipmr_can_free_table()` (lines 17–20) that relies solely on `!check_net(net)`. This check fails to guarantee that the multicast routing table is freed when the network subsystem is uninitialized. If operations such as routing table reinitialization proceed under these conditions, the unreleased tables will conflict with newly allocated resources, directly triggering a kernel bug. The patch addresses this issue with a unified helper function, `mr_can_free_table()`, with a more robust condition: `!check_net(net) || !net_initialized(net)` (lines 7–10). This change ensures the table is only freed under safe conditions, eliminating the kernel bug across all configurations.

The detection of this bug demonstrates the coordination between two of Pacaller’s core components. First, the static configuration analysis component enabled the vulnerable code path, making it accessible to the fuzzer. Subsequently, guided by the generated syscall specifications, Pacaller called network-related syscalls such as `unshare()` to manipulate network namespaces during their setup and teardown phases. This specific sequence triggered the conditions where the routing table was not correctly released, leading to a resource allocation conflict that exposed the kernel bug.

Case 2: Packet-Syscall Coordinated Triggered Bug. To illustrate the effectiveness of combining packet injection with syscall execution, we present a kernel bug (Bug #11) in the network scheduling module, as shown in Fig. 12. This bug, fixed by maintainers, is in the kernel’s SKBPRIO scheduler (`net/sched/sch_skbprio.c`) and manifests when SKBPRIO operates as a child qdisc under the Token Bucket Filter (TBF). The flaw stems from an overly strict assertion in `skbprio_enqueue()` (line 6). To perform traffic shaping, TBF occasionally “peeks” at packets in its child qdisc without dequeuing them when tokens are insufficient. This action creates a transient inconsistency in the queue length counters between the parent and child qdiscs, which the `BUG_ON` assertion misinterprets as a fatal error, triggering a kernel bug. The patch removes this overly restrictive assertion.

The discovery of this bug highlights a key limitation in conventional fuzzing: while a fuzzer might establish the necessary TBF and SKBPRIO configuration through syscalls, it typically lacks the precise control over packet timing and volume needed to trigger this bug. Pacaller overcomes this with its packet-syscall coordinated fuzzing. First, Pacaller uses syscalls to establish the required qdisc hierarchy, placing an SKBPRIO qdisc under a TBF. Subsequently, Pacaller injects

```

1 --- a/net/sched/sch_skbprio.c
2 +++ b/net/sched/sch_skbprio.c
3 @@ -123,8 +123,6 @@ static int skbprio_enqueue(struct sk_buff
4     *skb, struct Qdisc *sch,
5     // Check to update highest and lowest priorities.
6     if (skb_queue_empty(lp_qdisc)) {
7         if (q->lowest_prio == q->highest_prio) {
8             // The incoming packet is the only packet in queue.
9             // Bug: Strict assertion fails due to TBF peek
10             BUG_ON(sch->q.len != 1);
11             q->lowest_prio = prio;
12             q->highest_prio = prio;
13         } else {
14             @@ -156,7 +154,6 @@ static struct sk_buff
15             *skbprio_dequeue(struct Qdisc *sch)
16             // Update highest priority field.
17             if (skb_queue_empty(hpq)) {
18                 if (q->lowest_prio == q->highest_prio) {
19                     // Bug: Another strict assertion on queue length
20                     BUG_ON(sch->q.len == 0);
21                     q->highest_prio = 0;
22                     q->lowest_prio = SKBPRIO_MAX_PRIORITY - 1;
23                 } else {

```

Fig. 12. The patch that fixes a kernel bug in the network scheduler by removing overly strict assertions.

a sustained burst of traffic to deplete the TBF’s tokens. This sequence reliably induces the “peek” behavior, reproduces the queue length inconsistency, and exposes the deep flaw in the scheduler’s logic.

Answer to RQ1: Pacaller proved highly effective in bug detection, discovering 26 previously unknown bugs in the Linux kernel network stack, of which 16 were confirmed and 10 have been fixed. These findings validate its robust bug-finding capabilities.

6.3. Performance of pacaller

To answer RQ2, we evaluated the fuzzing performance of Pacaller against five state-of-the-art kernel fuzzers: Syzkaller, SyzGen++, KernelGPT, TCP-Fuzz, and TRON. We selected Syzkaller as the benchmark. SyzGen++ and KernelGPT were selected to evaluate the resource-driven input generation technique of Pacaller. SyzGen++ automatically infers syscall specifications from execution traces, while KernelGPT utilizes large language models to generate specifications. TCP-Fuzz and TRON were selected to evaluate the coordinated fuzzing strategy of Pacaller. TCP-Fuzz employs a dependency-based strategy to guide test case generation. As TCP-Fuzz targets user-level protocols using scripts, we implemented a basic packet injection mechanism within the Syzkaller framework and ported the core dependency generation strategy of TCP-Fuzz for a fair comparison. We refer to this variant as Syz-TCP. TRON employs a feedback-guided strategy to guide test case generation. We measured performance based on two key metrics: code coverage and the number of unique crashes within the network stack. To ensure a fair comparison, we configured all fuzzers to specifically target the network subsystem. For the fuzzers based on the Syzkaller framework, we utilized the `focus_areas` feature to prioritize test cases that explore the `net/` and `drivers/net/` directories.

Code Coverage. The detailed coverage results for the network stack are presented in Table 5. The data shows that Pacaller consistently achieves the highest code coverage across all tested kernel versions. Specifically, Pacaller achieves 32849.5, 32529.2, 33623.7, and 33548.3 coverage within the network stack of Linux v6.11, v6.13, v6.15, and v6.16, averaging 33137.7. Pacaller demonstrates superior performance across all comparison groups. First, compared to the benchmark, Pacaller achieves 22.3% higher network stack coverage than Syzkaller. Second, regarding the resource-driven input generation, Pacaller outperforms SyzGen++ by 20.2% and KernelGPT by 18.3%. Third, regarding the coordinated fuzzing strategy, Pacaller achieves 15.0% higher coverage than Syz-TCP and 13.3% higher than TRON.

Table 5
Code coverage of Pacaller compared with state-of-the-art fuzzers.

Version	Pacaller	Syzkaller	SyzGen++	KernelGPT	Syz-TCP	TRON
6.11	32849.5	27108.8 (+21.2%)	27871.1 (+17.9%)	27957.4 (+17.5%)	28689.5 (+14.5%)	29121.8 (+12.8%)
6.13	32529.2	26161.6 (+24.3%)	26652.5 (+22.1%)	27221.1 (+19.5%)	28042.4 (+16.0%)	28534.4 (+14.0%)
6.15	33623.7	27512.1 (+22.2%)	27996.9 (+20.1%)	28494.6 (+18.0%)	29288.9 (+14.8%)	29755.5 (+13.0%)
6.16	33548.3	27583.6 (+21.6%)	27774.6 (+20.8%)	28382.7 (+18.2%)	29248.7 (+14.7%)	29584.0 (+13.4%)
Average	33137.7	27091.5 (+22.3%)	27573.8 (+20.2%)	28013.9 (+18.3%)	28817.4 (+15.0%)	29248.9 (+13.3%)

Table 6
Unique crashes in the network stack detected by Pacaller compared with state-of-the-art fuzzers.

Version	Pacaller	Syzkaller	SyzGen++	KernelGPT	Syz-TCP	TRON
6.11	11.8	4.4 (2.68×)	4.5 (2.62×)	4.9 (2.41×)	7.1 (1.66×)	7.6 (1.55×)
6.13	11.3	4.5 (2.51×)	4.2 (2.69×)	4.6 (2.46×)	6.5 (1.74×)	7.2 (1.57×)
6.15	12.2	4.2 (2.90×)	4.8 (2.54×)	5.1 (2.39×)	7.4 (1.65×)	8.1 (1.51×)
6.16	10.6	3.2 (3.31×)	3.1 (3.42×)	3.5 (3.03×)	6.2 (1.71×)	6.9 (1.54×)
Average	11.5	4.1 (2.82×)	4.2 (2.77×)	4.5 (2.54×)	6.8 (1.69×)	7.5 (1.54×)

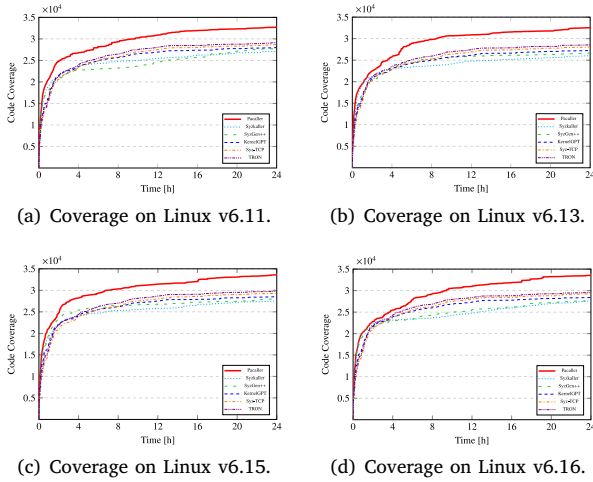


Fig. 13. Code coverage growth of Pacaller, Syzkaller, SyzGen++, KernelGPT, Syz-TCP and TRON on Linux v6.11, v6.13, v6.15, v6.16.

These results demonstrate that Pacaller significantly surpasses these state-of-the-art tools.

Fig. 13 depicts the code coverage growth curves for six fuzzers over a 24-hour period. A common trend observed is an initial rapid increase in code coverage for all fuzzers within the first few hours, after which the growth rate begins to stabilize. Notably, throughout the fuzzing process, Pacaller grows faster and consistently achieves significantly higher overall coverage compared to all other fuzzers. This superior performance stems from two of Pacaller's core innovations. First, the automated extraction of network resources and subsequent generation of syscall specifications and packet models enables Pacaller to generate more effective test payloads. Second, our coordinated fuzzing method combines syscall execution with packet injection, allowing Pacaller to trigger complex states that are inaccessible to syscall-based fuzzers.

Unique crashes. We also analyzed the network stack crashes reported by each fuzzer across the four kernel versions. As summarized in [Table 6](#), Pacaller consistently outperforms all other fuzzers. On average, Pacaller identified 11.5 unique crashes over 24 h. Compared to the benchmark, Pacaller outperforms Syzkaller (4.1 crashes) by a factor of 2.82. Regarding resource-driven input generation, Pacaller outperforms SyzGen++ (4.2 crashes) by a factor of 2.77 and KernelGPT (4.5 crashes) by a factor of 2.54. Regarding the coordinated fuzzing strategy, Pacaller outperforms Syz-TCP (6.8 crashes) by a factor of 1.69 and TRON (7.5 crashes) by a factor of 1.54.

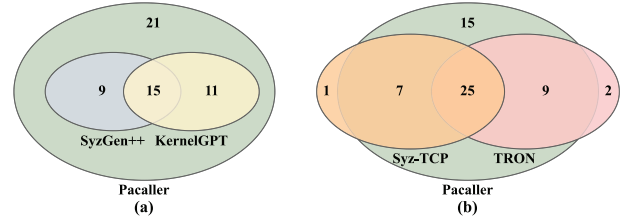


Fig. 14. Venn diagrams of unique crashes: (a) Pacaller, SyzGen++, and KernelGPT; (b) Pacaller, Syz-TCP, and TRON.

To demonstrate the overall bug-finding capability, we aggregated the unique crashes detected across 40 fuzzing campaigns (i.e., 10 repetitions on each of the 4 kernel versions) of each tool. [Fig. 14](#) presents the Venn diagrams of these accumulated crashes. Pacaller detected a total of 56 unique crashes. Notably, Pacaller detected all 21 bugs found by Syzkaller. We omitted Syzkaller from [Fig. 14](#) for visual clarity. As shown in [Fig. 14\(a\)](#), the crashes detected by SyzGen++ (24 crashes) and KernelGPT (26 crashes) are subsets of the crashes detected by Pacaller. Pacaller detected 21 bugs that these syscall-only tools missed. As shown in [Fig. 14\(b\)](#), Pacaller covers most bugs found by Syz-TCP (33 crashes) and TRON (36 crashes). Moreover, Pacaller detected 15 bugs that other coordinated fuzzers missed. These significant gains underscore the effectiveness of our coordinated fuzzing strategy. By coordinating syscall execution with packet injection, Pacaller triggers complex interactions within the network stack, thereby exposing bugs inaccessible to conventional fuzzers.

Answer to RQ2: Pacaller outperforms all state-of-the-art fuzzers on both key metrics. On average, it achieves 13.3% to 22.3% higher code coverage and detects 1.54×–2.82× more unique crashes than other tools. Furthermore, Pacaller finds most bugs reported by other tools plus unique bugs they miss.

6.4. Ablation study

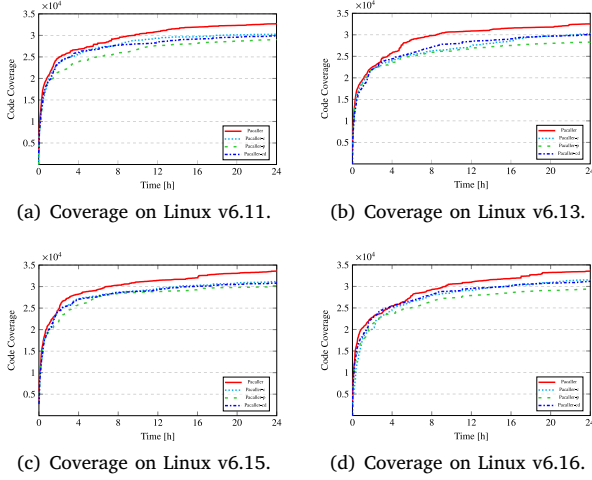
To answer RQ3, we conducted an ablation study by creating three variants of Pacaller to quantify the specific contribution of each component:

- Pacaller-c, which removes our generated syscall specifications. This variant uses the packet component and the coordinated fuzzing strategy but relies on the standard baseline syscall specifications from Syzkaller.

Table 7

Code coverage of Pacaller and its variants.

Version	Pacaller	Pacaller-c	Pacaller-cd	Pacaller-p
6.11	32849.5	30267.0 (+8.5%)	30022.6 (+9.4%)	29107.7 (+12.9%)
6.13	32529.2	30480.3 (+6.7%)	30185.4 (+7.8%)	28385.6 (+14.6%)
6.15	33623.7	31213.6 (+7.7%)	30950.2 (+8.6%)	30224.0 (+11.2%)
6.16	33548.3	31537.7 (+6.4%)	31280.1 (+7.3%)	29517.2 (+13.7%)
Average	33137.7	30874.7 (+7.3%)	30609.6 (+8.3%)	29308.6 (+13.1%)

**Fig. 15.** Code coverage growth of Pacaller, Pacaller-c, Pacaller-p and Pacaller-cd on Linux v6.11, v6.13, v6.15, v6.16.

- Pacaller-cd, which removes the directed coordinated fuzzing strategy. This variant retains both the generated specifications and the packet component but injects packets randomly without the guidance of our coordinated fuzzing.
- Pacaller-p, which removes the packet injection component. The coordinated fuzzing strategy is also removed because it depends on packet injection. This variant relies on an extended set of specifications, which includes both the baseline specifications from Syzkaller and our generated network-specific specifications.

We then compared the code coverage achieved and unique crashes detected by Pacaller and these variants. We designed three comparisons to evaluate the impact of each component. First, comparing Pacaller with Pacaller-c demonstrates the contribution of the generated syscall specifications. Second, comparing Pacaller with Pacaller-cd demonstrates the contribution of the coordinated fuzzing strategy. Third, comparing Pacaller-cd with Pacaller-p demonstrates the contribution of the packet injection component.

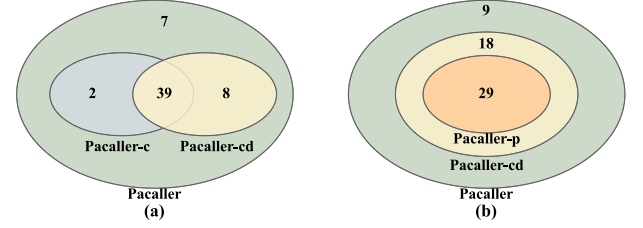
Code Coverage. The effectiveness of Pacaller's core components is detailed through a comparative analysis of network stack coverage, with statistics presented in Table 7 and growth curves in Fig. 15. Pacaller consistently achieves the highest coverage, averaging 33137.7 across all evaluated kernel versions. Its variants show distinct tiers: Pacaller-c averages 30874.7, Pacaller-cd averages 30609.6, and Pacaller-p averages 29308.6. The curves confirm Pacaller's leading trajectory; all variants grow rapidly before stabilizing.

To quantify the contribution of each component, we performed pairwise comparisons. First, Pacaller achieves 7.3% higher coverage than Pacaller-c, confirming that generated syscall specifications guide the fuzzer to explore code paths inaccessible to baseline syscalls. Second, although both versions possess the same syscall specifications and packet injection capabilities, Pacaller achieves 8.3% higher coverage than Pacaller-cd, confirming that the coordination strategy is critical for triggering complex network states, whereas random packet injection and syscall execution is insufficient. Finally, Pacaller-cd achieves

Table 8

Unique crashes in the network stack detected by Pacaller, Pacaller-c, Pacaller-cd and Pacaller-p.

Version	Pacaller	Pacaller-c	Pacaller-cd	Pacaller-p
6.11	11.8	9.4 (1.26×)	10.2 (1.16×)	7.6 (1.55×)
6.13	11.3	9.6 (1.18×)	9.9 (1.14×)	7.4 (1.53×)
6.15	12.2	10.9 (1.12×)	11.0 (1.11×)	8.5 (1.44×)
6.16	10.6	8.3 (1.28×)	9.2 (1.15×)	7.3 (1.45×)
Average	11.5	9.6 (1.20×)	10.1 (1.14×)	7.7 (1.49×)

**Fig. 16.** Venn diagrams of total unique crashes detected by Pacaller and its variants. (a) Comparison among Pacaller, Pacaller-c, and Pacaller-cd. (b) Comparison among Pacaller, Pacaller-cd, and Pacaller-p.

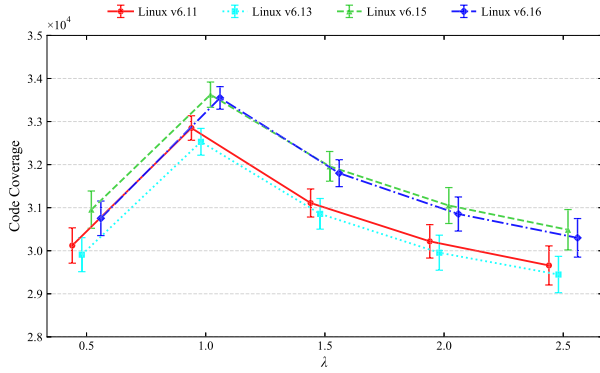
4.4% higher coverage than Pacaller-p, confirming that packet injection complements syscall-only fuzzing even without coordination, although the effectiveness is limited without the coordination strategy.

Unique Crashes. To quantify the impact of Pacaller's core components on its bug detection capabilities, we compared the number of unique network stack crashes discovered by Pacaller and its variants, as summarized in Table 8. On average, Pacaller identified 11.5 unique crashes over 24 h, outperforming all variants. First, comparing Pacaller with Pacaller-c shows that the generated syscall specifications contribute to 1.9 additional crash detections on average (1.20× improvement). Second, comparing Pacaller with Pacaller-cd shows that the coordinated fuzzing strategy contributes to 1.4 additional crash detections on average (1.14× improvement). Finally, comparing Pacaller-cd with Pacaller-p shows that the packet injection component contributes to 2.4 additional crash detections (1.31× improvement) even without coordination.

We also aggregated the total unique crashes detected across all fuzzing campaigns, as shown in Fig. 16. Pacaller detected a total of 56 unique crashes. Fig. 16(a) illustrates the contribution of the generated syscall specifications and the coordination strategy. First, comparing Pacaller (56 crashes) with Pacaller-c (41 crashes) shows the contribution of syscall specifications. Pacaller detected 15 additional bugs (8 and 7 in Fig. 16(a)), showing that generated specifications reach bugs inaccessible to baseline specifications. Second, comparing Pacaller with Pacaller-cd (47 crashes) shows the contribution of the coordination strategy. Pacaller detected 9 additional bugs (2 and 7 in Fig. 16(a)), indicating that random packet injection and syscall execution is insufficient. Notably, Pacaller detected 7 crashes that were missed by both Pacaller-c and Pacaller-cd. This result proves that triggering these 7 bugs requires the combination of generated syscall specifications and the coordination strategy. Fig. 16(b) illustrates the contribution of the packet injection component. Comparing Pacaller-cd with Pacaller-p shows the contribution of packet injection. Pacaller-p detected 29 crashes using only syscalls. With the addition of packet injection, Pacaller-cd detected 18 additional crashes (47 total). This result confirms that packet injection is essential for testing the network stack.

Table 9Code coverage with different λ values.

Version	$\lambda = 0.5$	$\lambda = 1.0$	$\lambda = 1.5$	$\lambda = 2.0$	$\lambda = 2.5$
v6.11	30 121.3	32 849.5	31 108.6	30 217.8	29 658.6
v6.13	29 908.5	32 529.2	30 856.4	29 953.9	29 447.3
v6.15	30 954.2	33 623.7	31 959.2	31 049.5	30 486.8
v6.16	30 752.9	33 548.3	31 801.1	30 852.7	30 299.0
Average	30 434.2	33 137.7	31 431.3	30 518.5	29 972.9

**Fig. 17.** Code coverage sensitivity to λ on four kernels.

Answer to RQ3: The ablation study confirms the contribution of each component. Each component individually improves the code coverage (7.3% from syscall specifications, 8.3% from coordination strategy, and 4.4% from packet injection) and bug detection (1.9 crashes from syscall specifications, 1.4 crashes from coordination strategy, and 2.4 crashes from packet injection). These results demonstrate that the combination of these components is effective.

6.5. Parameter sensitivity analysis

To answer RQ4, to determine the optimal hyper-parameter λ , we executed Pacaller with different values on four Linux kernel versions. The parameter λ controls the weight of the code coverage feedback from the target network directories. We tested five evenly spaced values (0.5, 1.0, 1.5, 2.0, 2.5), executing each configuration for 24 h with 10 repetitions.

Table 9 lists the detailed code coverage results. Fig. 17 visualizes the average coverage trends with error bars. The results show a distinct inverted U-shaped trend across all kernel versions. The code coverage peaks at $\lambda = 1.0$. Specifically, Pacaller achieves the highest average coverage of 33,137.7 with $\lambda = 1.0$. The coverage decreases when λ is lower or higher than 1.0. A lower value of 0.5 reduces the average coverage to 30,434.2 because it provides insufficient guidance toward the network stack. Conversely, a higher value of 2.5 results in the lowest average coverage of 29,972.9 as it overly restricts exploration to specific network paths. Furthermore, the coverage trend stabilizes as λ increases, indicating that larger values have similar restrictive effects. These findings demonstrate that $\lambda = 1.0$ effectively balances focused testing and general exploration. Consequently, we use $\lambda = 1.0$ as Pacaller's default.

Answer to RQ4: The sensitivity analysis shows that $\lambda = 1.0$ achieves the highest code coverage. Lower values lack network guidance, while higher values restrict exploration. Consequently, $\lambda = 1.0$ effectively balances focused testing with general exploration.

7. Discussion

We discuss Pacaller's generality and limitations.

Generality. The core methodology of Pacaller, which combines resource-driven input generation and coordinated fuzzing, is general. The approach can be applied to any system that exposes functionality through complex data structures and accepts multiple input types. Applying Pacaller to a new target, such as the network stack of another operating system, requires adapting two components that directly interface with the operating system. First, the payload execution component within the C executor needs to be modified to support the target's syscall interface and packet injection mechanisms. Second, the runtime state monitoring component must be modified to collect internal kernel events on the new platform.

Limitation. Although Pacaller proves effective, we acknowledge several limitations that suggest avenues for future work. First, our packet component does not simulate the low-level behavior of physical network devices, such as DMA operations or hardware offloading. Future work could incorporate device emulation to uncover bugs in driver-hardware interactions. Second, the scope of our resource identification is limited to the primary network directories, potentially missing related code in other subsystems like fs/ or mm/ that interact indirectly with the network stack. Future work could broaden this scope to further improve coverage, though this would require careful analysis to maintain efficiency.

8. Related work

Configuration Analysis. Configuration analysis is crucial for maintaining the quality of highly configurable systems, and research continues to examine its complexity and influence on security bugs. Prior research explores static means of improving configuration-based analysis. For example, C-Reconfigurator (Schubert et al., 2022), SugarC (Patterson et al., 2022), and Maki (Pappas and Gazzillo, 2024) demonstrate the feasibility of statically rewriting applications' configuration-dependent code to instead be invokable at runtime, such as rewriting an `#ifdef`-wrapped code block to be guarded by an `if` statement. Beyond analysis techniques, generating effective configurations for testing is also a key problem. Early work (Yan et al., 2017) includes using combinatorial testing to cover option interactions, while ConfigFuzz (Zhang et al., 2023) targets fuzzing command-line options of user-space programs. More recently, to address patch coverage in continuous integration, krepair (Yildiran et al., 2024) was developed to automatically repair configurations. The work by (Hasanov et al., 2025) further shows that integrating krepair into continuous kernel fuzzing workflows significantly improves bug detection.

Kernel Fuzzing. Kernel fuzzing aims to test the kernel's API surface through syscalls. State-of-the-art fuzzers, such as Syzkaller (Vyukov and Konovalov, 2015a), generate sequences of syscalls to explore kernel behavior. To improve the fuzzing process, some research focuses on better seed selection strategies (Pailoor et al., 2018), while other work uses reinforcement learning to more effectively explore the kernel state space (Wang et al., 2021). A significant body of research focuses on automatically generating syscall specifications from source code or binaries, with notable examples including SyzGen (Chen et al., 2021), SyzGen++ (Chen et al., 2024), KernelGPT (Yang et al., 2025), Difuze (Corina et al., 2017), and SyzDescribe (Hao et al., 2023). Another line of research develops directed fuzzing techniques, such as SyzDirect (Tan et al., 2023), to steer fuzzing toward specific code locations like newly added patches. Yet these methods operate primarily through syscalls.

Protocol Fuzzing. In a separate domain, protocol fuzzers test how systems handle network traffic by sending packets. Traditional grammar-based tools like Peach (Tech, 2013), Sulley (OpenRCE, 2014), and Spike (Aitel, 2017) generate packet sequences from manually-defined protocol formats. To overcome the limitations of manual modeling, modern protocol fuzzing has incorporated automated techniques.

Some approaches, like AFLNet (Pham et al., 2020), SNPSFuzzer (Li et al., 2022) and StateAFL (Natella, 2022), infer and test the behavior of stateful protocols through state machine learning. Other research adapts fuzzing to specific domains, such as Internet of Things (IoT) devices (Chen et al., 2018; Feng et al., 2021). These tools excel at packet-level analysis but operate from user space and cannot coordinate the specific kernel configurations required to trigger many bugs.

Coordinated Fuzzing. Recognizing the limitations of applying kernel fuzzing and protocol fuzzing in isolation, recent research has begun to explore coordinated fuzzing approaches that combine both syscall execution and packet injection. TCP-Fuzz (Zou et al., 2021) is an early example that generates coordinated inputs for the TCP stack, but its scope is limited and it relies on differential testing. EmNetTest (Amusuo et al., 2023) targets embedded network stacks by setting protocol states with syscalls before injecting packets. Nyx-Net (Schumilo et al., 2022) utilizes snapshot-based fuzzing to target user-space protocols by emulating network functions, intentionally bypassing the actual kernel network stack to improve performance. TRON (Zhang, 2025) synthesizes payloads for the Linux kernel using offline trace analysis, but TRON combines inputs based on static patterns without runtime state guidance. This principle extends to testing device drivers, where Dr.Fuzz (Zhao et al., 2022) models device states to coordinate sequences of syscalls and hardware interactions. These pioneering works demonstrate the value of coordination, but they are often tailored to specific protocols, device drivers, or testing goals. A systematic and automated coordinated fuzzing methodology for the kernel network stacks remains an open research direction. Pacaller addresses this gap through coordinated testing of syscall-packet interactions.

9. Conclusion

In this paper, we presented Pacaller, a novel fuzzer designed to address challenges in Linux kernel network stack testing. Pacaller integrates three key techniques: automated static configuration analysis to enhance code reachability; systematic resource extraction to generate both protocol-aware syscall specifications and packet models; and a packet-syscall coordinated fuzzing methodology that coordinates syscall execution with packet injection. We evaluated Pacaller on four recent versions of the Linux kernel, discovering 26 previously unknown bugs (16 confirmed, 10 fixed) and achieving 13.3% to 22.3% higher network stack coverage than Syzkaller, SyzGen++, KernelGPT, Syz-TCP, and TRON. These results demonstrate that Pacaller significantly improves network stack testing.

CRedit authorship contribution statement

Guoyu Yin: Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Yuhan Chen:** Software, Methodology, Conceptualization. **Changjian Liu:** Visualization, Investigation, Conceptualization. **Weibo Zhang:** Writing – review & editing, Validation, Project administration. **Fan Ding:** Resources, Project administration, Data curation. **Dalong Shi:** Supervision, Software, Resources. **Yuheng Shen:** Visualization, Supervision, Methodology. **Heyuan Shi:** Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported in part by the National Natural Science Foundation of China (No. 62472448), the Alibaba Innovative Research Program, the High Performance Computing Center of Central South University, and the Frontier Interdisciplinary Direction Quantum Technology Project of Central South University.

Data availability

Research Link Provided.

References

- Aitel, D., 2017. Spike fuzzer. URL <https://github.com/guilhermeferreira/spikepp>.
- Amusuo, P.C., Méndez, R.A.C., Xu, Z., Machiry, A., Davis, J.C., 2023. Systematically detecting packet validation vulnerabilities in embedded network stacks. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, pp. 926–938. <http://dx.doi.org/10.1109/ASE56229.2023.00095>.
- Bavendiek, S., 2021. Attack Surface Analysis of the Linux Kernel Based on Complexity Metrics Master's Thesis. study course Applied Informatics/Software Engineering.
- Bellard, F., 2005. Qemu, a fast and portable dynamic translator. In: USENIX Annual Technical Conference, FREENIX Track. Vol. 41, California, USA, pp. 10–5555.
- Bhurltel, M., Rawat, D.B., 2023. Unveiling the landscape of operating system vulnerabilities. *Futur. Internet* 15 (7), 248.
- Chen, J., Diao, W., Zhao, Q., Zuo, C., Lin, Z., Wang, X., Lau, W., Sun, M., Yang, R., Zhang, K., 2018. Iotfuzzer: Discovering memory corruptions in iot through app-based fuzzing. <http://dx.doi.org/10.14722/ndss.2018.23166>.
- Chen, W., Hao, Y., Zhang, Z., Zou, X., Kirat, D., Mishra, S., Schales, D., Jang, J., Qian, Z., 2024. Syzgen++: Dependency inference for augmenting kernel driver fuzzing. pp. 4661–4677. <http://dx.doi.org/10.1109/SP54263.2024.00269>.
- Chen, W., Wang, Y., Zhang, Z., Qian, Z., 2021. Syzgen: Automated generation of syscall specification of closed-source macos drivers. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. CCS '21, Association for Computing Machinery, New York, NY, USA, pp. 749–763. <http://dx.doi.org/10.1145/3460120.3484564>.
- Corina, J., Machiry, A., Salls, C., Shoshitaishvili, Y., Hao, S., Kruegel, C., Vigna, G., 2017. Difuze: Interface aware fuzzing for kernel drivers. In: 24th ACM SIGSAC Conference on Computer and Communications Security, CCS 2017. CCS '17, ACM, New York, NY, USA, pp. 2123–2138. <http://dx.doi.org/10.1145/3133956.3134069>.
- Eisele, M., Ebert, D., Huth, C., Zeller, A., 2023. Fuzzing embedded systems using debug interfaces. In: 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023. ISSTA 2023, ACM, New York, NY, USA, pp. 1031–1042. <http://dx.doi.org/10.1145/3597926.3598115>.
- Feng, X., Sun, R., Zhu, X., Xue, M., Wen, S., Liu, D., Nepal, S., Xiang, Y., 2021. Snipuzz: Black-box fuzzing of iot firmware via message snippet inference. In: 28th ACM SIGSAC Conference on Computer and Communications Security, CCS 2021. CCS '21, ACM, New York, NY, USA, pp. 337–350. <http://dx.doi.org/10.1145/3460120.3484543>.
- Fiterau-Brostean, P., Jonsson, B., Merget, R., De Ruiter, J., Sagonas, K., Somorovsky, J., 2020. Analysis of (DTLS) implementations using protocol state fuzzing. In: 29th USENIX Security Symposium. USENIX Security 20, pp. 2523–2540.
- Google, 2014. Kernel address sanitizer. <https://www.kernel.org/doc/html/latest/dev-tools/kasan.html>.
- Google, 2019. Kernel concurrency sanitizer. <https://www.kernel.org/doc/html/latest/dev-tools/kcsan.html>.
- Hao, Y., Li, G., Zou, X., Chen, W., Zhu, S., Qian, Z., Sani, A.A., Principled, title=Static Generation of Syscall Descriptions for Kernel Drivers, 2023. 2023 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, Los Alamitos, CA, USA, pp. 3262–3278. <http://dx.doi.org/10.1109/SP46215.2023.10179298>, URL <https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.10179298>.
- Hasanov, S., Nagy, S., Gazzillo, P., 2025. A little goes a long way: Tuning configuration selection for continuous kernel fuzzing. In: 2025 IEEE/ACM 47th International Conference on Software Engineering. ICSE, pp. 795–807. <http://dx.doi.org/10.1109/ICSE55347.2025.00042>.
- Jeong, D.R., Kim, K., Shivakumar, B., Lee, B., Shin, I., 2019. Razzar: Finding Kernel Race Bugs through Fuzzing. In: 40th IEEE Symposium on Security and Privacy. SP 2019, IEEE, pp. 754–768, URL <http://dblp.uni-trier.de/db/conf/sp/sp2019.html#JeongKSL19>.
- Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, Association for Computing Machinery, New York, NY, USA, pp. 2123–2138. <http://dx.doi.org/10.1145/3243734.3243804>.
- lcamtuf, 2013. American fuzzy lop. <https://lcamtuf.coredump.cx/afl/>.
- Li, J., Li, S., Sun, G., Chen, T., Yu, H., 2022. Snpsfuzzer: A fast greybox fuzzer for stateful network protocols using snapshots. *IEEE Trans. Inf. Forensics Secur.* 17, 2673–2687. <http://dx.doi.org/10.1109/TIFS.2022.3192991>.
- Luo, Z., Yu, J., Du, Q., Zhao, Y., Wu, F., Shi, H., Chang, W., Jiang, Y., 2024. Parallel fuzzing of iot messaging protocols through collaborative packet generation. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 43 (11), 3431–3442. <http://dx.doi.org/10.1109/TCAD.2024.3444705>.
- Luo, Z., Zuo, F., Shen, Y., Jiao, X., Chang, W., Jiang, Y., 2020. Ics protocol fuzzing: Coverage guided packet crack and generation. In: 2020 57th ACM/IEEE Design Automation Conference. DAC, pp. 1–6. <http://dx.doi.org/10.1109/DAC18072.2020.9218603>.
- Manes, V.J., Han, H., Han, C., Cha, S.K., Egele, M., Schwartz, E.J., Woo, M., 2018. Fuzzing: Art, science, and engineering. arXiv preprint [arXiv:1812.00140](https://arxiv.org/abs/1812.00140).

- McDonald, L., Haq, M.I.U., Barkworth, A., 2021. Survey of software fuzzing techniques. *Natella, R., 2022. Stateaf: Greybox fuzzing for stateful network servers. Empir. Softw. Eng.* 27 (7), 191.
- National Vulnerability Database, 2025. CVE-2025-21703 Detail. URL <https://nvd.nist.gov/vuln/detail/cve-2025-21703>.
- OpenRCE, 2014. Sulley fuzzing framework. URL <https://github.com/OpenRCE/sulley>.
- Pailoor, S., Aday, A., Jana, S., 2018. MoonShine: Optimizing OS fuzzer seed selection with trace distillation. In: 27th USENIX Security Symposium. USENIX Security 18, USENIX Association, Baltimore, MD, pp. 729–743. URL <https://www.usenix.org/conference/usenixsecurity18/presentation/pailoor>.
- Pappas, B., Gazzillo, P., 2024. Semantic analysis of macro usage for portability. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA, <http://dx.doi.org/10.1145/3597503.3623323>.
- Patterson, Z., Zhang, Z., Pappas, B., Wei, S., Gazzillo, P., 2022. Sugarc: Scalable desugaring of real-world preprocessor usage into pure c. In: 2022 IEEE/ACM 44th International Conference on Software Engineering. ICSE, pp. 2056–2067. <http://dx.doi.org/10.1145/3510003.3512763>.
- Pham, V.-T., Böhme, M., Roychoudhury, A., 2020. Aflnet: A greybox fuzzer for network protocols. In: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification. ICST, IEEE, pp. 460–465.
- Qian, L., Carpenter, B.E., 2012. A flow-based performance analysis of tcp and tcp applications. In: 2012 18th IEEE International Conference on Networks. IEEE, ICON, pp. 41–45.
- Schubert, P.D., Gazzillo, P., Patterson, Z., Braha, J., Schiebel, F., Hermann, B., Wei, S., Boddien, E., 2022. Static data-flow analysis for software product lines in c: Revoking the preprocessor's special role. *Autom. Softw. Engg.* 29 (1), <http://dx.doi.org/10.1007/s10515-022-00333-1>.
- Schumilo, S., Aschermann, C., Jemmett, A., Abbasi, A., Holz, T., 2022. Nyx-net: network fuzzing with incremental snapshots. In: Proceedings of the Seventeenth European Conference on Computer Systems. EuroSys '22, Association for Computing Machinery, New York, NY, USA, pp. 166–180. <http://dx.doi.org/10.1145/3492321.3519591>.
- Shen, Y., Liu, J., Xu, Y., Sun, H., Wang, M., Guan, N., Shi, H., Jiang, Y., 2024. Enhancing ros system fuzzing through callback tracing. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 76–87.
- Shen, Y., Sun, H., Jiang, Y., Shi, H., Yang, Y., Chang, W., 2021. Rtkaller: State-Aware Task Generation for RTOS Fuzzing. *ACM Trans. Embed. Comput. Syst.* 20 (5s), <http://dx.doi.org/10.1145/3477014>.
- Shi, H., Chen, S., Wang, R., Chen, Y., Zhang, W., Zhang, Q., Shen, Y., Shi, X., Hu, C., Jiang, Y., 2024. Industry practice of directed kernel fuzzing for open-source linux distribution. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. pp. 2159–2169.
- Shi, H., Zhang, W., Wang, R., Shi, X., Yin, G., Chen, S., Chen, Y., Zhang, Q., Liu, J., Shen, Y., 2025. Dragonradar: Fuzzing linux kernel deployed in cloud-native environment. In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 829–833.
- SimonKagstrom, 2015. Kcov. <https://github.com/SimonKagstrom/kcov>.
- Tan, X., Zhang, Y., Lu, J., Xiong, X., Liu, Z., Yang, M., 2023. Syzdirect: Directed greybox fuzzing for linux kernel. In: 30th ACM SIGSAC Conference on Computer and Communications Security. CCS 2023, ACM, pp. 1630–1644. <http://dx.doi.org/10.1145/3576915.3623146>.
- Teich, P., 2013. Peach fuzzer. URL <https://gitlab.com/peachtech/peach-fuzzer-community>.
- Vyukov, D., Konovalov, A., 2015a. Syzkaller: An unsupervised coverage-guided kernel fuzzer. URL <https://github.com/google/syzkaller>.
- Vyukov, D., Konovalov, A., 2015b. Syzlang: System call description language. URL https://github.com/google/syzkaller/blob/master/docs/syscall_descriptions_syntax.md.
- Wang, J., Yu, L., Luo, X., 2024. Llmif: Augmented large language model for fuzzing iot devices. In: 45th IEEE Symposium on Security and Privacy. SP 2024, IEEE Computer Society, Los Alamitos, CA, USA, pp. 881–896. <http://dx.doi.org/10.1109/SP54263.2024.00211>, URL <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00211>.
- Wang, D., Zhang, Z., Zhang, H., Qian, Z., Krishnamurthy, S.V., Abu-Ghazaleh, N., 2021. SyzVegas: Beating kernel fuzzing odds with reinforcement learning. In: 30th USENIX Security Symposium. USENIX Security 21, USENIX Association, pp. 2741–2758, URL <https://www.usenix.org/conference/usenixsecurity21/presentation/wang-daimeng>.
- Xu, M., Kashyap, S., Zhao, H., Kim, T., 2020. Krace: Data race fuzzing for kernel file systems. In: 41th IEEE Symposium on Security and Privacy. SP 2020, pp. 1643–1660. <http://dx.doi.org/10.1109/SP40000.2020.00078>.
- Yan, D., Pan, L., Yan, R., Yan, J., Zhang, J., 2017. Comprehensive static analysis for configurable software via combinatorial instantiation. In: 2017 IEEE 41st Annual Computer Software and Applications Conference. COMPSAC, Vol. 1, pp. 67–74. <http://dx.doi.org/10.1109/COMPSAC.2017.91>.
- Yang, C., Zhao, Z., Zhang, L., 2025. Kernelgpt: Enhanced kernel fuzzing via large language models. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. Vol. 2, pp. 560–573.
- Yildiran, N.F., Oh, J., Lawall, J., Gazzillo, P., 2024. Maximizing patch coverage for testing of highly-configurable software without exploding build times. *Proc. ACM Softw. Eng.* 1 (FSE), <http://dx.doi.org/10.1145/3643746>.
- Yu, J., Luo, Z., Xia, F., Zhao, Y., Shi, H., Jiang, Y., 2024. Spfuzz: Stateful path based parallel fuzzing for protocols in autonomous vehicles. In: Proceedings of the 61st ACM/IEEE Design Automation Conference. DAC '24, Association for Computing Machinery, <http://dx.doi.org/10.1145/3649329.3658270>.
- Zhang, Q., 2025. Tron: Automated fuzzing framework. URL <https://github.com/rushwow/TRON>.
- Zhang, Z., Klees, G., Wang, E., Hicks, M., Wei, S., 2023. Fuzzing configurations of program options. *ACM Trans. Softw. Eng. Methodol.*
- Zhang, Q., Shen, Y., Liu, J., Xu, Y., Shi, H., Jiang, Y., Chang, W., 2024. Ecg: Augmenting embedded operating system fuzzing via llm-based corpus generation. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 43 (11), 4238–4249. <http://dx.doi.org/10.1109/TCAD.2024.3447220>.
- Zhao, W., Lu, K., Wu, Q., Qi, Y., 2022. Semantic-informed driver fuzzing without both the hardware devices and the emulators. In: Network and Distributed Systems Security (NDSS) Symposium 2022.
- Zheng, Y., Davanian, A., Yin, H., Song, C., Zhu, H., Sun, L., 2019. FIRM-AFL: High-Throughput greybox fuzzing of IoT firmware via augmented process emulation. In: 28th USENIX Security Symposium. USENIX Security 19, USENIX Association, Santa Clara, CA, pp. 1099–1114, URL <https://www.usenix.org/conference/usenixsecurity19/presentation/zheng>.
- Zou, Y.-H., Bai, J.-J., Zhou, J., Tan, J., Qin, C., Hu, S.-M., 2021. Tcp-fuzz: Detecting memory and semantic bugs in tcp stacks with fuzzing. In: 2021 USENIX Annual Technical Conference. USENIX ATC 21, pp. 489–502.
- Zuo, F., Luo, Z., Yu, J., Chen, T., Xu, Z., Cui, A., Jiang, Y., 2022. Vulnerability detection of ics protocols via cross-state fuzzing. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (11), 4457–4468. <http://dx.doi.org/10.1109/TCAD.2022.3201471>.